DevSecOps Pipeline: Node.js Shopping Cart Application Deployment to AKS

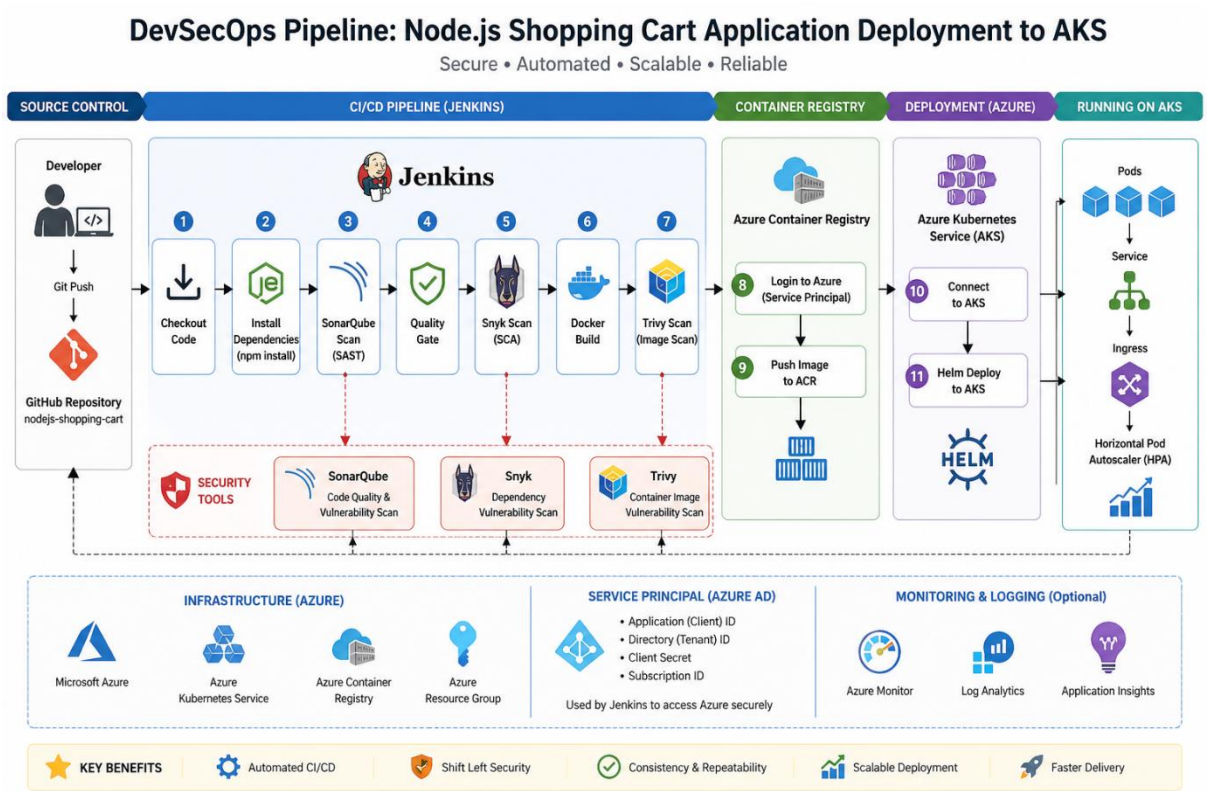
Overview

This project demonstrates a complete **DevSecOps pipeline** using:

- Jenkins (CI/CD)
- SonarQube (SAST)
- Snyk (SCA)
- Trivy (Container Security)
- Docker (Containerization)
- Azure Container Registry (ACR)
- Azure Kubernetes Service (AKS)
- Helm (Deployment)

Architecture Flow

Developer → Git Push → Jenkins Trigger → Checkout Code
→ Install Dependencies
→ SonarQube (SAST) → Quality Gate
→ Snyk (SCA)
→ Docker Build
→ Trivy Image Scan
→ Login to Azure & ACR
→ Push Image to ACR
→ Connect to AKS
→ Helm Deploy to AKS
→ Pods → Service → Ingress → HPA



Prerequisites

1. Active Azure Account
2. Snyk Account
3. Ubuntu VM (Minimum 2 CPUs)
4. Open all required ports

Application Repository

https://github.com/luckyncpl/nodejs-shopping-cart

This repository includes:

- Node.js application
- Dockerfile
- Helm charts

Security Tools Explanation

Snyk SCA means Software Composition Analysis.

It checks your project dependencies like express, lodash, body-parser, etc. from package.json / package-lock.json and tells you:

- Which packages have vulnerabilities
- Severity: low, medium, high, critical
- Which dependency is causing the issue
- Suggested fix or upgrade version
- License issues, if any

Snyk CLI snyk test checks open-source vulnerabilities and license issues in your project

Trivy :

What is Trivy?

Trivy is a security scanning tool used to detect vulnerabilities in container images, application dependencies, and configuration files.

Why is it used?

It helps identify security risks such as outdated packages or known vulnerabilities before the application is deployed. This reduces the chance of deploying insecure software.

When is it used?

It is typically used after building a Docker image and before pushing it to a registry or deploying it to a cluster.

What does it check?

- Operating system packages inside the image
- Application dependencies (like npm packages)
- Configuration issues (Dockerfile, Kubernetes, etc.)

Simple understanding

Trivy ensures that the container image you are about to deploy does not contain known security issues.

Azure Service Principal Creation (Simple Steps)

Step 1: Create App Registration

1. Go to **Microsoft Entra ID (Azure AD)**
2. Click **App registrations**
3. Click **New registration**
4. Give a name → *example: lucky*
5. Click **Register**

Step 2: Create Secret

1. Open the created app (*lucky*)
2. Go to **Certificates & Secrets**
3. Click **New client secret**
4. Click **Add**
5. Copy and save the **Value** (important)

Step 3: Note Required Details

From the app overview, copy:

- Application (Client) ID
- Directory (Tenant) ID
- Object ID
- Client Secret Value
- Subscription ID

Step 4: Assign Role (Access to Azure Resources)

1. Go to **Subscriptions**
2. Select your subscription
3. Click **Access Control (IAM)**
4. Go to **Role Assignments**
5. Click **+ Add** → **Add role assignment**

Step 5: Configure Role

1. Select Role → **Owner**
2. Click **Next**

Step 6: Assign to App

1. Click **Select members**
2. Search for your app (*lucky*)
3. Select it
4. Click **Next**

Step 7: Complete Assignment

1. Review details
2. Click **Review + Assign**

Final Result

Now you have:

- Service Principal created
- Permissions assigned
- Credentials ready for Jenkins

Environment Setup (Ubuntu VM)

1. Install Java

```
sudo apt update
sudo apt install fontconfig openjdk-21-jre -y
java -version
```

---

2. Install Jenkins

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key

echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update
sudo apt install jenkins -y
```

---

3. Install Docker

```
sudo apt update -y
wget -O docker.sh https://get.docker.com/
sudo sh docker.sh

sudo usermod -aG docker jenkins
sudo usermod -aG docker $USER

sudo systemctl restart docker
sudo systemctl restart jenkins

newgrp docker
docker --version
```

---

4. Install Trivy

```
sudo apt update
sudo apt-get install wget apt-transport-https gnupg lsb-release -y

wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | sudo apt-key add -

echo deb https://aquasecurity.github.io/trivy-repo/deb $(lsb_release -sc) main | sudo tee -a
/etc/apt/sources.list.d/trivy.list

sudo apt-get update
sudo apt-get install trivy -y

trivy --version
```

---

5. Run SonarQube (Docker)

```
sudo docker pull sonarqube
sudo docker run -d -p 9000:9000 sonarqube
```

Access:
http://<VM-IP>:9000
Default login:
username: admin
password: admin

---

6. Install kubectl

sudo apt update
sudo snap install --classic kubectl
kubectl version --client

---

7. Install Helm
sudo apt update
curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
helm version

---

8. Install Azure CLI
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
az --version
Login:
az login --use-device-code

---

9. Fix Required Library
sudo apt install -y libatomic1
sudo systemctl restart jenkins

Jenkins Setup
Plugins Required

- Pipeline Stage View
- NodeJS Plugin
- SonarQube Scanner Plugin

Tools Configuration
Go to:
Manage Jenkins → Global Tool Configuration
Add:

- NodeJS → Name: nodejs
- Sonar Scanner → Name: sonar-scanner

Note: Don't change Names here

SonarQube Configuration
Manage Jenkins → Configure System
Add:

- Name: SonarQube
- URL: http://<VM-IP>:9000
- Token: Add via credentials

Credentials Setup in Jenkins
Add the following:

1. **SonarQube Token**
2. **Snyk Token**
3. **Azure Service Principal**
    - Client ID
    - Client Secret
    - Tenant ID

For detailed steps : https://github.com/luckysuie/Pipelines/blob/main/Pipeline-14/README.md (**Azure cloud credentials configuration)**

Azure Infrastructure Setup
Create ACR
az acr create --resource-group lucky --name luckyregistry11 --sku Basic

---

Create AKS Cluster
az aks create --resource-group lucky --name lucky-aks-cluster11 --node-count 1 --generate-ssh-keys --attach-acr luckyregistry11

GitHub Webhook Setup

1. Fork repository
2. Go to:

Settings → Webhooks → Add webhook

3. Configure:

- Payload URL:

http://<VM-IP>:8080/github-webhook/

- SSL: Disable
- Event: Push

Jenkins Pipeline Setup
Create Pipeline

- New Item → Pipeline
- Select:

Pipeline script from SCM

- Provide:
    - GitHub Repo URL
    - Branch (main/master)

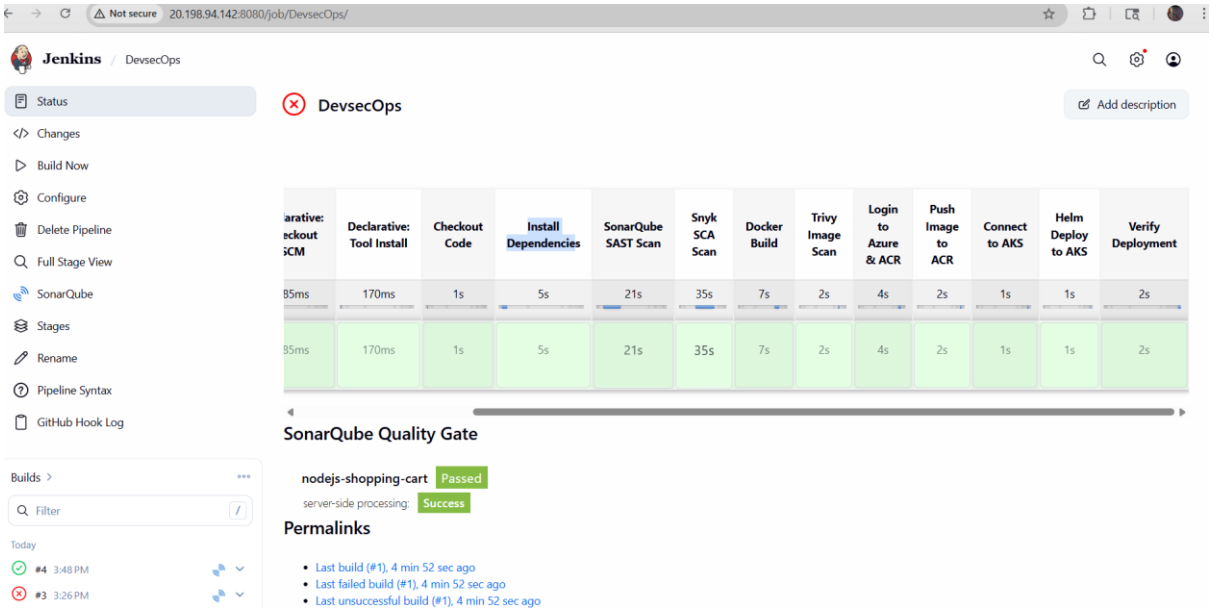
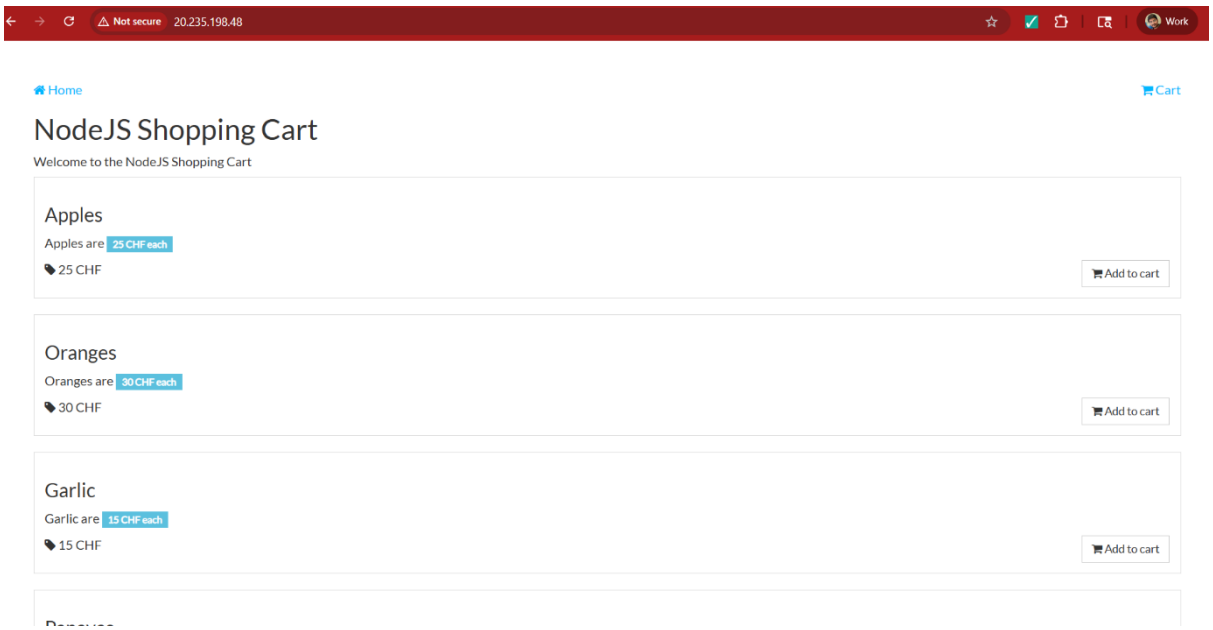
Enable Trigger
GitHub hook trigger for GITScm polling

Pipeline Stages

- clone your repository into local and open it in visual studio code

1. Checkout Code
2. Install Dependencies
3. SonarQube SAST Scan
4. Snyk SCA Scan
5. Docker Build
6. Trivy Image Scan
7. Login to Azure & ACR
8. Push Image to ACR
9. Connect to AKS
10. Helm Deploy to AKS
11. Verify Deployment

Result :



Application: